

BERT Fine-tuning Report

HONG Yuxiang
yhongbb@connect.ust.hk

Abstract

This project focuses on the performance optimization of the BERT model in the text classification task – sentiment analysis with the IMDB dataset. Through 10 sets of control experiments, it systematically explores the impact of the model parameter configuration, training strategies, and structural improvements on the text classification performance. The experiments cover fine-tuning the whole BERT with users' default parameters, adjusting the amount of data, parameter freezing (full freezing / partial freezing), and optimizing hyperparameters such as learning rate, batch size, number of epochs, maximum text truncation length, dropout rate, and using the output embeddings of other tokens instead of that of the class token. The results show that full parameter fine-tuning combined with reasonable hyperparameters (such as a learning rate of $1e-5$, batch size of 8, and 3-6 training epochs) can achieve the best classification performance (test accuracy range $\approx 89\%-92\%$); the small amount of data negatively affects the model's performance; freezing strategies and custom model designs need to be carefully designed to avoid performance loss; also, the direct use of non-[CLS] tokens requires structural adjustments (such as manually calculating the loss). This project provides practical references for parameter optimization and structural improvement in the practical application of BERT in text classification tasks.

Introduction

Text classification is a basic task in Natural Language Processing (NLP), and it is widely used in scenarios like sentiment analysis, spam detection, and news classification. BERT (Bidirectional Encoder Representations from Transformers), as a representation of pretrained language models, has shown its strong transfer learning capabilities in text classification tasks by learning bidirectional context semantic representations on large-scale corpora. However, the BERT model, which was fine-tuned with the users' default parameters and using the [CLS] token as the classification feature, may not exhibit its great performance in all scenarios; it is still restricted by a few factors, such as the size of the training set, the learning rate, or even its own model structure.

This project aims at a binary text classification task, sentiment analysis with the IMDB dataset, based on the pretrained bert-base-uncased model provided by HuggingFace. Ten sets of experiments were designed to assist in exploring the following questions:

1. The standard performance benchmark of BERT with full parameter fine-tuning
2. The impact of using different amounts of data for training and testing
3. The impact of freezing BERT and only training the classification head (called linear probing)
4. The impact of freezing earlier layers of BERT and fine-tuning the last few layers
5. The impact of varying the learning rate of BERT
6. The impact of varying the batch size of BERT
7. The impact of varying the dropout rate of BERT
8. The impact of varying the number of epochs
9. The impact of varying the maximum text truncation length
10. The impact of using the output embeddings of other tokens instead of those of the class token

By systematically comparing the test accuracy and classification reports (precision/recall/F1 score) of each experiment group, the best practice strategies for BERT in test classification tasks are summarized.

Methodology

The project adopts a binary text classification task (category labels: 0 = negative, 1 = positive); the dataset used is the IMDB Dataset, which contains 50000 data samples. The dataset is randomly shuffled and split into a training set and a test set according to an 80% to 20% ratio. There are 40000 samples in the standard training set and 10000 samples in the standard test set.

The base model is HuggingFace's BertForSequenceClassification, loading the pretrained bert-base-uncased. The default classification logic is extracting the hidden state of the [CLS] token of the input sequence as the sequence representation, and inputting it into the classification head to obtain the category probability.

10 sets of experiments were designed, and each group adjusted one variable to compare their impact on the test accuracy and classification performance given by the classification report. The specific experimental configurations and objectives are as follows:

Experiment No.	Experiment Title	Adjustment of Core Variables	Expected Goal
1	Fine-tuning the entire BERT with users' default parameters	Full parameter fine-tuning (not freezing BERT), max text truncation length 128, learning rate 2e-5, batch size 4, epoch 3	Establish benchmark performance and verify the effectiveness of BERT fine-tuning
2	Using different amounts of data for training and testing	Training data volume was reduced from 40000 to 400, test data volume was reduced from 10000 to 100	Analyze the impact of data volume on the BERT model
3	Freezing BERT and only training the classification head (this is called linear probing)	Freezing all parameters of BERT (only train the classification header), other configurations are as same as Experiment 1's	Comparing the effect differences between parameter freezing and full fine-tuning
4	Varying the learning rate	Adjusting the learning rate to a relatively lower value, 1e-5	Exploring the influence of the learning rate on the BERT model
5	Varying the batch size	Adjusting the batch size from 4 to 8	Analyzing the influence of batch size on the BERT model
6	Varying number of epochs	Increasing the number of epochs from 3 to 6	Analyzing the impact of the number of epochs on the performance of the BERT model
7	Freezing earlier layers of BERT and fine-tuning the last few layers	Freezing the first 6 layers of BERT, fine-tuning the classification headers of the last few layers, and the other configurations are the same as in Experiment 1	Explore the local optimization strategy of parameter freezing
8	Varying the maximum text truncation length	Adjusting the max text truncation length from 128 to 256	Analyzing the influence of text truncation length on the model's capture of context information
9	Varying the dropout rate	Adjusting the dropout rate from 0.1 to 0.3	Evaluating the inhibitory effect of regularization intensity on overfitting
10	Using the output embeddings of other tokens instead of those of the class token	Use the mean pooling of all tokens instead of [CLS] as the classification feature to customize the model	Verifying the potential improvement of classification performance by non [CLS] tokens

Results & Analysis

The summarized results of each experiment are listed as follows:

Experiment No.	Experiment Title	Training Accuracy	Test Accuracy	Final test accuracy rate & Comparison with Baseline Experiment
1	Fine-tuning the entire BERT with users' default parameters	epoch1: 0.8566 epoch2: 0.9260 epoch3: 0.9620	epoch1: 0.8863 epoch2: 0.8935 epoch3: 0.8883	88.83%
2	Using different amounts of data for training and testing	epoch1: 0.6650 epoch2: 0.8675 epoch3: 0.9500	epoch1: 0.7700 epoch2: 0.8000 epoch3: 0.6800	68%(-23.45%)
3	Freezing BERT and only training the classification head (this is called linear probing)	epoch1: 0.5869 epoch2: 0.6527 epoch3: 0.6758	epoch1: 0.6634 epoch2: 0.6916 epoch3: 0.7000	70%(-21.20%)
4	Varying the learning rate	epoch1: 0.8666 epoch2: 0.9283 epoch3: 0.9672	epoch1: 0.8650 epoch2: 0.9013 epoch3: 0.8911	89.11%(+0.32%)
5	Varying the batch size	epoch1: 0.8633 epoch2: 0.9302 epoch3: 0.9689	epoch1: 0.8927 epoch2: 0.8960 epoch3: 0.8925	89.25%(+0.47%)
6	Varying number of epochs	epoch1: 0.8594 epoch2: 0.9282 epoch3: 0.9641 epoch4: 0.9789 epoch5: 0.9849 epoch6: 0.9867	epoch1: 0.8868 epoch2: 0.8905 epoch3: 0.8873 epoch4: 0.8888 epoch5: 0.8851 epoch6: 0.8893	88.93%(+0.11%)
7	Freezing earlier layers of BERT and fine-tuning the last few layers	epoch1: 0.8653 epoch2: 0.9265 epoch3: 0.9634	epoch1: 0.8944 epoch2: 0.8953 epoch3: 0.8883	88.83%(0.00%)
8	Varying the maximum text truncation length	epoch1: 0.8909 epoch2: 0.9445 epoch3: 0.9714	epoch1: 0.9203 epoch2: 0.9173 epoch3: 0.9208	92.08%(+3.66%)

9	Varying the dropout rate	epoch1: 0.8418 epoch2: 0.8827 epoch3: 0.9073	epoch1: 0.8642 epoch2: 0.8774 epoch3: 0.8908	89.08%(+0.28%)
10	Using the output embeddings of other tokens instead of those of the class token	epoch1: 0.8639 epoch2: 0.9278 epoch3: 0.9654	epoch1: 0.8757 epoch2: 0.8961 epoch3: 0.8889	88.89%(+0.07%)

(Experiment 1 works as the baseline and was highlighted with green color; Experiments with the highest and lowest test accuracy were separately highlighted by red color and blue color, respectively. These two sets of experiments also had the greatest fluctuations above and below the baseline experiment.)

Experiment 1 (Fine-tuning the entire BERT with users' default parameters):

Configurations: Full-parameter fine-tuning, learning rate 2e-5, max text truncation length 128, batch size 4, epoch 3, training sample size 40000, test sample size 10000

results:

- Test Accuracy: There was a slight decline in the test accuracy, but the overall stability is around 89%.
- Classification Report (epoch 3): The precision rate, recall rate, and F1 value of negative/positive at epoch 3 are all around 0.89; macro avg and weighted avg are both 0.89, demonstrating that the model's classification ability for positive and negative classes is balanced.

Analysis:

As a benchmark experiment, full-parameter fine-tuning shows the strong performance of BERT in text classification tasks, with a stable test accuracy rate of around 89% and no significant deviation between positive and negative classes. It can be used as a comparison baseline for subsequent experiments.

Experiment 2 (Using different amounts of data for training and testing):

Configurations: learning rate 2e-5, max text truncation length 128, batch size 4, epoch 3, training sample size 400, test sample size 100

results:

- Test Accuracy: The test accuracy is significantly overfitting.

- Classification Report (epoch 3): The recall rate of negative is as high as 1.00 (almost all predictions are negative), but the precision rate is only 0.60. The precision rate of positive is 1.00, but the recall rate is only 0.38. macro avg=0.65, and the performance has decreased significantly.

Analysis:

When the data volume is not sufficient, the model can't learn robust features of representations; overfitting (the high recall rate of positive at epoch 3) and underfitting (the overall accuracy is only 68%) are intended to appear, which validates the importance of large-scale data for BERT fine-tuning.

Experiment 3 (Freezing BERT and only training the classification head (this is called linear probing):

Configurations: Freezing all BERT parameters, others are as same as Experiment 1's

results:

- Test Accuracy: The test accuracy is finally stabilized at around 70%.
- Classification Report (epoch 3): The F1 score for negative/positive is 0.67-0.73, and its macro avg is 0.70, which is significantly lower than Experiment 1's 0.89.

Analysis:

Freezing BERT parameters significantly lowers the model's performance, which indicates that the pre-trained features of BERT need to be fine-tuned to adapt to downstream tasks.

Experiment 4 (Varying the learning rate):

Configurations: learning rate $1e-5$, the others are the same as in Experiment 1.

results:

- Test Accuracy: The test accuracy reaches the peak at epoch 2, then has a slight decline in epoch 3.
- Classification Report (epoch 2): The precision, recall rates, and F1 score for negative/positive are around 0.90, macro avg reaches 0.90, which overperforms epoch 3's result in Experiment 1 (0.89).

Analysis:

A lower learning rate ($1e-5$) makes the training more stable and achieves a higher test accuracy at epoch 2 (90.13%). The final test accuracy (89.11%) is close to that in Experiment 1, which validates that the lower learning rate has a positive effect on model convergence.

Experiment 5 (Varying the batch size):

Configurations: batch size changes from 4 to 8, others are as same as in Experiment 1

results:

- Test Accuracy: The test accuracy has a high value at epoch 1, then it reaches the peak, and finally stabilizes.
- Classification Report (epoch 2): The F1 scores for negative/positive are both 0.90; the macro avg is relatively higher compared with Experiment 1's macro avg.

Analysis:

Increasing the batch size may enhance the stability of gradient updates, reduce training noise, and enable the model to achieve a high accuracy rate at epoch 2. The final accuracy rate (89.25%) is still higher than the benchmark of Experiment 1 (88.83%).

Experiment 6 (Varying number of epochs):

Configurations: the number of epoch changes from 3 to 6, others are as same as in Experiment 1

results:

- Test Accuracy: The test accuracy is stabilized at around 89%.
- Classification Report: The macro avg of each epoch remained at 0.89; there is no performance reduction during the process.

Analysis:

Increasing the number of epochs to 6 did not significantly improve the accuracy rate, but it maintained the stability, showing that 3-epoch training is sufficiently convergent; too many epochs might result in diminishing marginal benefits.

Experiment 7 (Freezing earlier layers of BERT and fine-tuning the last few layers):

Configurations: Freezing the first 6 layers, fine-tuning the last few layers, others are as same as in Experiment 1

results:

- Test Accuracy: The test accuracy stabilized at around 89%.
- Classification Report (epoch 3): The F1 scores for negative/positive are 0.88-0.90, macro avg is 0.89, which is close to the benchmark of Experiment 1.

Analysis:

Partial freezing strategy (freezing the first few layers) reduced the number of parameters needed to be fine-tuned with almost no performance loss, which balances the training efficiency and effectiveness.

Experiment 8 (Varying the maximum text truncation length):

Configurations: Adjusting the maximum text truncation length from 128 to 256, others are as same as in Experiment 1

results:

- Test Accuracy: The test accuracy stabilizes at around 92%.
- Classification Report (epoch 3): The F1 scores for negative/positive are both 0.92, macro avg is 0.92, which overperforms all the rest model variants.

Analysis:

Increasing the maximum text truncation length enables the model to capture much longer contextual information, which significantly improves the model's performance. It also points out that the text length is one of the core hyperparameters that impact BERT performance, especially for long text tasks.

Experiment 9 (Varying the dropout rate):

Configurations: Adjusting the dropout rate from the default 0.1 to 0.3, others are as same as in Experiment 1

results:

- Test Accuracy: The test accuracy gradually increased to 89%.
- Classification Report (epoch 3): The F1 scores for negative/positive are both 0.89, macro avg is 0.89, which is close to Experiment 1's. The loss is gradually reduced.

Analysis:

Since the adjustment range in this experiment was relatively small, its impact on the model's performance was reflected in the gradually reduced loss (the training performance is stable).

Experiment 10 (Using the output embeddings of other tokens instead of those of the class token):

Configurations: Use the mean pooling of all tokens instead of [CLS] as the classification feature to customize the model

results:

- Test Accuracy: The test accuracy reaches the peak at epoch 2, then stabilizes at around 89%.
- Classification Report (epoch 3): The F1 scores for negative/positive are 0.88-0.89, macro avg is 0.89, which is close to the benchmark of Experiment 1.

Analysis:

Using the representations of non-[CLS] tokens, such as Mean Pooling, can achieve performance similar to that of [CLS]. But when designing the custom model, passing labels needs to be avoided, and loss needs to be manually calculated.

Discussion

Based on these 10 sets of experiments, we can find that:

1. Data volume is crucial for model training. Large-scale data (40000 training data and 10000 test data) is the precondition for BERT to show great performance on text classification tasks; a small amount of data (400 training data and 100 test data) might cause overfitting or underfitting, and test accuracy is reduced to 68%.
2. By comparing Experiment 1 and Experiment 3, it shows that full parameter fine-tuning (without freezing BERT parameters) can achieve a test accuracy of 89% under reasonable hyperparameters (learning rate $2e-5$, batch size 4, maximum text truncation length 128), which is significantly better than freezing parameters (70%).
3. The space for hyperparameter tuning is significant:
 - a. A low learning rate ($1e-5$) and a large batch size (8) can improve the stability of the BERT model.
 - b. Increasing the maximum text truncation length (128 to 256) has the most significant improvement in the model's performance (test accuracy achieves 92.08%, which is the highest in all these ten experiments), indicating the maximum text truncation length is a key factor.
4. Experiment 10 verified that the token representations (such as the mean pooling) can serve as an alternative to [CLS], but it needs to be implemented through a custom model; however, labels should not be passed into the model, and loss needs to be manually calculated during the process.

Conclusion

This project systematically analyzed factors that impact the BERT's performance on text classification tasks based on these 10 sets of experiments, and we can get conclusions as follow:

1. Full parameters fine-tuning combined with reasonable hyperparameters is the optimal benchmark configuration in text classification tasks.
2. Data volume and maximum text truncation length are two key factors that significantly affect the BERT model's performance.
3. Freezing BERT parameters in this specific task (the sentiment analysis) causes the obvious performance loss; while the performances of partial freezing and the representations of

non-[CLS] token demonstrate that these two strategies can be leveraged as alternative solutions for some text classification tasks.

4. Hyperparameters like the learning rate, the batch size, and the dropout rate, have a significant impact on the convergence and final performance of the model.

This project provides a complete practical path from benchmark verification to parameter optimization for the actual application of BERT in text classification tasks. Subsequently, it can be further extended to multi-classification tasks, long text scenarios, or comparative studies with other models.

Appendix

Images of training accuracy and test accuracy trends for each experiment:







